

Appendix: Complete R Code for FINSTAD Case 1

CRUZ, Ricardo Miguel Iñigo GALEDO, Enrique Lorenzo Hermoso
SEBALLOS, Josiah Dweyn Panganiban SEECHUNG, Camille Castro
SISON, Aaron Joshua Estacio

Appendix A1 - Setup & Library Import

Loads the required R libraries (dplyr, tidyr, lubridate, ggplot2, etc.) and defines global helper functions used throughout the analysis. The working directory is resolved automatically to ensure data file paths work regardless of whether the script runs from the project root or the scripts/ folder.

```
# Load necessary libraries
library(dplyr)
library(tidyr)
library(lubridate)
library(DBI)
library(RSQLite)
library(ggplot2)
library(scales)
library(kableExtra)

# Safe table formatter for PDF to prevent overlapping columns and
#   overflowing margins
safe_kable <- function(data, caption = NULL, digits = 4) {
  knitr::kable(data, caption = caption, digits = digits, format = "latex",
    ↪ booktabs = TRUE) %>%
    kable_styling(font_size = 8, latex_options = c("scale_down",
      ↪ "HOLD_position")) %>%
    column_spec(1:ncol(data), width_min = "1.5cm")
}

# Set working directory to project root so data paths resolve consistently
# Ensure data paths resolve correctly regardless of working directory
if (dir.exists("data")) {
  data_dir <- "data"
} else if (dir.exists("../data")) {
```

```

    data_dir <- "../data"
  } else {
    stop("Cannot find data/ directory. Run from project root or scripts/.")
  }

# Global options
options(stringsAsFactors = FALSE, scipen = 999, width = 72)

# Helper to parse BDO volume strings (e.g. "2.50M" to 2500000)
parse_bdo_volume <- function(x) {
  x <- gsub(",", "", x)
  mult <- ifelse(grepl("M", x, ignore.case = TRUE), 1e6,
                 ifelse(grepl("K", x, ignore.case = TRUE), 1e3, 1))
  x_num <- as.numeric(gsub("[A-Za-z%]", "", x))
  x_num * mult
}

```

Appendix A2 - Ticker Definitions

Defines the asset ticker symbols used throughout the case: AAPL (US equity), BDO.PS (Philippine bank stock), and BTC-USD (cryptocurrency).

```

# Define tickers
us_asset <- "AAPL"
ph_asset <- "BDO.PS"
crypto <- "BTC-USD"

```

Appendix A3 - Data Loading (GALEDO, Enrique Lorenzo Hermoso)

Loads all 8 CSV datasets from the data/ directory into R data frames. Records the original observation count for each dataset before any cleaning operations. Datasets include 4 asset price series (AAPL, BDO, BTC, SPY) and 4 FRED macroeconomic series (CPI, DGS10, FEDFUNDS, VIX).

```

# Data Loading -- GALEDO, Enrique Lorenzo Hermoso (Sec 3: Data Acquisition)

# Load all 8 CSVs from the data directory
aapl <- read.csv(file.path(data_dir, "AAPL.csv"), stringsAsFactors = FALSE)
bdo  <- read.csv(file.path(data_dir, "BDO.csv"), stringsAsFactors = FALSE,
  ↪   fileEncoding = "UTF-8-BOM")
btc  <- read.csv(file.path(data_dir, "BTC.csv"), stringsAsFactors = FALSE)
spy  <- read.csv(file.path(data_dir, "SPY.csv"), stringsAsFactors = FALSE)

```

```

cpi      <- read.csv(file.path(data_dir, "CPI.csv"), stringsAsFactors =
  ↪ FALSE)
dgs10    <- read.csv(file.path(data_dir, "DGS10.csv"), stringsAsFactors =
  ↪ FALSE)
fedfunds <- read.csv(file.path(data_dir, "FEDFUNDS.csv"), stringsAsFactors =
  ↪ FALSE)
vix      <- read.csv(file.path(data_dir, "VIX.csv"), stringsAsFactors =
  ↪ FALSE)

# Record original observations before any processing
obs_original <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX"),
  Original_Obs = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
    nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix))
)

safe_kable(obs_original, caption="Original Observations per Dataset")

```

Table 1: Original Observations per Dataset

Dataset	Original_Obs
AAPL	2512
BDO	2441
BTC	3652
SPY	2514
CPI	952
DGS10	16105
FEDFUNDS	863
VIX	9216

Appendix A4 - Data Cleaning (SISON, Aaron Joshua Estacio)

Standardises date formats across all 8 datasets, filters observations to the 2020-01-01 to 2025-12-31 sample period, counts missing values and duplicates, and computes daily returns for each OHLC asset.

```

# Sec 4: Data Cleaning -- SISON, Aaron Joshua Estacio

# Standardise date formats
# AAPL, BTC, SPY: POSIXct-style strings like "2016-06-30 00:00:00-04:00"
aapl <- aapl |> mutate(Date = as.Date(ymd_hms(Date)))
btc  <- btc  |> mutate(Date = as.Date(ymd_hms(Date)))
spy  <- spy  |> mutate(Date = as.Date(ymd_hms(Date)))

```

```

# BDO: MM/DD/YYYY
bdo <- bdo |> mutate(Date = mdy(Date))

# CPI, DGS10, FEDFUNDS, VIX: YYYY-MM-DD
cpi      <- cpi      |> mutate(Date = ymd(Date))
dgs10    <- dgs10    |> mutate(Date = ymd(Date))
fedfunds <- fedfunds |> mutate(Date = ymd(Date))
vix      <- vix      |> mutate(Date = ymd(Date))

# Filter ALL datasets to 2020-01-01 to 2025-12-31
start_date <- as.Date("2020-01-01")
end_date   <- as.Date("2025-12-31")

aapl <- aapl |> filter(Date >= start_date & Date <= end_date)
bdo  <- bdo  |> filter(Date >= start_date & Date <= end_date)
btc  <- btc  |> filter(Date >= start_date & Date <= end_date)
spy  <- spy  |> filter(Date >= start_date & Date <= end_date)
cpi   <- cpi   |> filter(Date >= start_date & Date <= end_date)
dgs10 <- dgs10 |> filter(Date >= start_date & Date <= end_date)
fedfunds <- fedfunds |> filter(Date >= start_date & Date <= end_date)
vix    <- vix    |> filter(Date >= start_date & Date <= end_date)

# Observations after filtering
obs_filtered <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX"),
  Filtered_Obs = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
    ↪ nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix))
)

# Missing values count
missing_summary <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX"),
  Missing_Values = c(sum(is.na(aapl)), sum(is.na(bdo)), sum(is.na(btc)),
    ↪ sum(is.na(spy)), sum(is.na(cpi)), sum(is.na(dgs10)),
    ↪ sum(is.na(fedfunds)), sum(is.na(vix)))
)

# Duplicate rows count
dup_summary <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX"),
  Duplicates = c(sum(duplicated(aapl)), sum(duplicated(bdo)),
    ↪ sum(duplicated(btc)),

```

```

        sum(duplicated(spy)), sum(duplicated(cpi)),
        ↪ sum(duplicated(dgs10)),
        sum(duplicated(fedfunds)), sum(duplicated(vix)))
    )

# Clean BDO: parse Vol. column if it exists
if ("Vol." %in% colnames(bdo)) {
  bdo <- bdo |> mutate(`Vol.` = parse_bdo_volume(`Vol.`))
}

# Calculate daily returns for OHLC assets
# Daily_Return = (Close / lag(Close)) - 1
# For BDO the Price column is the close price
aapl <- aapl |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close))
  ↪ - 1)
bdo <- bdo |> arrange(Date) |> mutate(Daily_Return = (Price / lag(Price))
  ↪ - 1)
btc <- btc |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close))
  ↪ - 1)
spy <- spy |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close))
  ↪ - 1)

# Note: BDO's Change. column can be used to verify Daily_Return computation
# The first observation of each asset will have NA for Daily_Return (no
  ↪ prior day)

# Combine cleaning summaries into one display table
clean_summary <- obs_filtered |>
  left_join(missing_summary, by = "Dataset") |>
  left_join(dup_summary, by = "Dataset")
safe_kable(clean_summary, caption="Observations After Filtering, Missing
  ↪ Values, and Duplicates")

```

Table 2: Observations After Filtering, Missing Values, and Duplicates

Dataset	Filtered_Obs	Missing_Values	Duplicates
AAPL	1508	0	0
BDO	1468	0	0
BTC	2192	0	0
SPY	1508	0	0
CPI	71	0	0
DGS10	1500	0	0
FEDFUNDS	72	0	0
VIX	1535	0	0

FRED data (CPI, DGS10, FEDFUNDS) has fewer rows due to monthly or business-daily

reporting frequency. Missing values introduced by join operations are handled in Section 5.

Appendix A5 - Database Integration (SISON, Aaron Joshua Estacio)

Demonstrates all four SQL join types (left, inner, full, anti) and justifies the chosen join strategy. Assembles the final analytical dataset by combining asset data with macroeconomic series. Exports the observation reduction table for inclusion in the main report.

```
# Sec 5: Database Integration -- SISON, Aaron Joshua Estacio

# Prepare asset data for joining
# Standardise column names: BDO uses Price (close price) and Vol. (volume)
bdo_clean <- bdo |> rename(Close = Price, Volume = `Vol.`)

# Add Asset identifier to each OHLC dataset and select common columns
aapl_clean <- aapl |> mutate(Asset = "AAPL") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
bdo_clean <- bdo_clean |> mutate(Asset = "BDO") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
btc_clean <- btc |> mutate(Asset = "BTC") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
spy_clean <- spy |> mutate(Asset = "SPY") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)

# Combine all assets into one long-format table
assets_all <- bind_rows(aapl_clean, bdo_clean, btc_clean, spy_clean)

# Pivot to wide format: one row per date, columns per asset
assets_wide <- assets_all |>
  select(Date, Asset, Close, Daily_Return) |>
  pivot_wider(names_from = Asset,
              values_from = c(Close, Daily_Return),
              names_sep = "_")

# Prepare macro datasets with descriptive column names
cpi_clean <- cpi |> rename(CPI = Value)
dgs10_clean <- dgs10 |> rename(DGS10 = Value)
fedfunds_clean <- fedfunds |> rename(FEDFUNDS = Value)
vix_clean <- vix |> rename(VIX = Value)

# Demonstration of all four join types
```

```

# 1) left_join: keep all asset dates, attach macro data where available
left_example <- assets_wide |>
  left_join(cpi_clean, by = "Date") |>
  left_join(dgs10_clean, by = "Date") |>
  left_join(fedfunds_clean, by = "Date") |>
  left_join(vix_clean, by = "Date")

# 2) inner_join: only dates present in ALL datasets
inner_example <- assets_wide |>
  inner_join(cpi_clean, by = "Date") |>
  inner_join(dgs10_clean, by = "Date") |>
  inner_join(fedfunds_clean, by = "Date") |>
  inner_join(vix_clean, by = "Date")

# 3) full_join: all dates from any dataset
full_example <- assets_wide |>
  full_join(cpi_clean, by = "Date") |>
  full_join(dgs10_clean, by = "Date") |>
  full_join(fedfunds_clean, by = "Date") |>
  full_join(vix_clean, by = "Date")

# 4) anti_join: asset dates with NO macro data
anti_example <- assets_wide |>
  anti_join(cpi_clean, by = "Date")

# Build the final analytical dataset
final_dataset <- left_example

# Observation reduction table

reduction_table <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX",
    "Assets (wide)", "Final Dataset"),
  Original_Obs = c(obs_original$Original_Obs, NA, NA),
  After_Cleaning = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
    ↪ nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix),
    nrow(assets_wide), NA),
  After_Joins = c(rep(NA, 8), NA, nrow(final_dataset))
)

knitr::kable(reduction_table, caption="Observation Counts Across Processing
  ↪ Stages")

```

Table 3: Observation Counts Across Processing Stages

Dataset	Original_Obs	After_Cleaning	After_Joins
AAPL	2512	1508	NA
BDO	2441	1468	NA
BTC	3652	2192	NA
SPY	2514	1508	NA
CPI	952	71	NA
DGS10	16105	1500	NA
FEDFUNDS	863	72	NA
VIX	9216	1535	NA
Assets (wide)	NA	2192	NA
Final Dataset	NA	NA	2192

```
library(knitr)
cat(kable(reduction_table, format = "latex", booktabs = TRUE), file =
  ↪  "../reports/sections/data_cleaning_table.tex")
```

Appendix A6 - Master Dataset Creation & SQL Database Setup (SISON, Aaron Joshua Estacio)

Builds the master analytical dataset by restructuring into long format (one row per asset-date combination) and saves it as a persistent SQLite database file. Pre-calculates CPI year-on-year change for inflation analysis. All subsequent SQL questions (A7-A16) query this persisted database.

```
# SQLite database setup - creates persisted master dataset

db_path <- "../data/master_dataset.sqlite"
unlink(db_path)
con <- dbConnect(RSQLite::SQLite(), db_path)

# Build long-format analytical dataset for SQL queries
# Each row is one asset-date combination with macro variables attached
sql_long <- bind_rows(
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
           Close = Close_AAPL, Daily_Return = Daily_Return_AAPL) |>
    mutate(Asset = "AAPL"),
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
```



```

        Close = Close_BDO, Daily_Return = Daily_Return_BDO) |>
mutate(Asset = "BDO"),
final_dataset |>
  select(Date, CPI, DGS10, FEDFUNDS, VIX,
        Close = Close_BTC, Daily_Return = Daily_Return_BTC) |>
mutate(Asset = "BTC"),
final_dataset |>
  select(Date, CPI, DGS10, FEDFUNDS, VIX,
        Close = Close_SPY, Daily_Return = Daily_Return_SPY) |>
mutate(Asset = "SPY")
)

# Pre-calculate CPI YoY for Q10 (inflation analysis)
# CPI is monthly; YoY change = (current / 12-month-ago) - 1
cpi_yoy <- cpi_clean |>
  arrange(Date) |>
  mutate(CPI_YoY = (CPI / lag(CPI, 12)) - 1) |>
  select(Date, CPI_YoY)

# Join CPI_YoY into the long dataset and forward-fill to cover all trading
  ↪ days
sql_long <- sql_long |>
  left_join(cpi_yoy, by = "Date") |>
  arrange(Date) |>
  tidyr::fill(CPI_YoY, .direction = "down")

dbWriteTable(con, "integrated_data", sql_long, overwrite = TRUE)
fin_data <- tbl(con, "integrated_data")

# Show first 5 rows of the master dataset as a preview
nrow_total <- collect(fin_data) |> nrow()
safe_kable(head(sql_long, 5), digits=4, caption=paste0("Master Dataset
  ↪ Preview (First 5 of ", nrow_total, " Rows)"))

```

Table 4: Master Dataset Preview (First 5 of 8768 Rows)

Date	CPI	DGS10	FEDFUNDS	VIX	Close	Daily_Return	Asset	CPI_YoY
2020-01-01	259.127	NA	1.55	NA	NA	NA	AAPL	NA
2020-01-01	259.127	NA	1.55	NA	NA	NA	BDO	NA
2020-01-01	259.127	NA	1.55	NA	7200.17	NA	BTC	NA
2020-01-01	259.127	NA	1.55	NA	NA	NA	SPY	NA
2020-01-02	NA	1.88	NA	12.47	72.33	NA	AAPL	NA

Appendix A7 - SQL Question 1: Risk-Adjusted Daily Return (SISON, Aaron Joshua Estacio)

Computes the ratio of mean daily return to standard deviation for each asset, producing a Sharpe-like measure of return efficiency. The corresponding bar chart is displayed in the Data Visualization section.

```
# Q1: Risk-Adjusted Daily Return by Asset -- SISON, Aaron Joshua Estacio

q1 <- fin_data |>
  group_by(Asset) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE)
  ) |>
  mutate(Risk_Adjusted_Return = Mean_Daily_Return / SD_Daily_Return) |>
  arrange(desc(Risk_Adjusted_Return)) |>
  collect()

spy_ratio <- q1 |> filter(Asset == "SPY") |> pull(Risk_Adjusted_Return)

safe_kable(q1, digits=4, caption="Q1 Data Output")
```

Table 5: Q1 Data Output

Asset	Mean_Daily_Return	SD_Daily_Return	Risk_Adjusted_Return
AAPL	0.0011	0.0200	0.0538
BTC	0.0017	0.0319	0.0520
SPY	0.0006	0.0131	0.0486
BDO	0.0003	0.0226	0.0131

Appendix A8 - SQL Question 2: COVID-19 Crash Cumulative Returns (SISON, Aaron Joshua Estacio)

Measures crisis resilience during the February to March 2020 COVID-19 market crash by computing cumulative returns for each asset.

```
# Q2: COVID-19 Crash Cumulative Returns -- SISON, Aaron Joshua Estacio

q2_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q2 <- q2_raw |>
```

```

filter(Date >= as.Date("2020-02-01") & Date <= as.Date("2020-03-31")) |>
filter(!is.na(Daily_Return)) |>
group_by(Asset) |>
arrange(Date) |>
mutate(Cumulative_Return = cumprod(1 + Daily_Return) - 1)

q2_summary <- q2 |>
  summarise(
    Start_Date = min(Date),
    End_Date = max(Date),
    Total_Cumulative_Return = last(Cumulative_Return),
    Observations = n()
  )

safe_kable(q2_summary, digits=4, caption="Q2 Summary Data Output")

```

Table 6: Q2 Summary Data Output

Asset	Start_Date	End_Date	Total_Cumulative_Return	Observations
AAPL	2020-02-03	2020-03-31	-0.1764	41
BDO	2020-02-03	2020-03-31	-0.3006	40
BTC	2020-02-01	2020-03-31	-0.3114	60
SPY	2020-02-03	2020-03-31	-0.1941	41

Appendix A9 - SQL Question 3: Pairwise Correlation of Daily Returns (SISON, Aaron Joshua Estacio)

Calculates the Pearson correlation coefficients for daily returns across all asset pairs and displays the correlation matrix.

```

# Q3: Pairwise Correlation of Daily Returns -- SISON, Aaron Joshua Estacio

returns_wide <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  pivot_wider(names_from = Asset, values_from = Daily_Return)

cor_matrix <- returns_wide |>
  select(AAPL, BDO, BTC, SPY) |>
  cor(use = "pairwise.complete.obs")

cor_long <- as.data.frame(cor_matrix) |>
  tibble::rownames_to_column("Asset1") |>
  tidyr::pivot_longer(-Asset1, names_to = "Asset2", values_to =
    ↪ "Correlation") |>

```

```
mutate(Asset1 = factor(Asset1, levels = rev(c("AAPL", "BDO", "BTC",
↪ "SPY"))),
      Asset2 = factor(Asset2, levels = c("AAPL", "BDO", "BTC", "SPY")))

safe_kable(round(cor_matrix, 4), caption="Q3 Correlation Matrix Output")
```

Table 7: Q3 Correlation Matrix Output

	AAPL	BDO	BTC	SPY
AAPL	1.0000	0.1044	0.2836	0.7839
BDO	0.1044	1.0000	0.0507	0.1235
BTC	0.2836	0.0507	1.0000	0.3825
SPY	0.7839	0.1235	0.3825	1.0000

Appendix A10 - SQL Question 4: Asset Ranking by Total Return (CRUZ, Ricardo Miguel Iñigo)

Ranks assets by total price return and annualized return over the full 2020-2025 sample period.

```
# Q4: Asset Ranking by Total Return -- CRUZ, Ricardo Miguel Inigo

q4_raw <- fin_data |>
  select(Date, Asset, Close, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q4 <- q4_raw |>
  filter(!is.na(Close), !is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  summarise(
    Start_Date = min(Date),
    End_Date = max(Date),
    Start_Close = first(Close),
    End_Close = last(Close),
    Total_Return = (End_Close / Start_Close) - 1,
    Annualized_Return = (1 + Total_Return)^(365.25 / as.numeric(End_Date -
↪ Start_Date)) - 1,
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
```

```

    arrange(desc(Total_Return))

q4_plot <- q4 |>
  mutate(Asset = factor(Asset, levels = q4$Asset[order(q4$Total_Return)]))

safe_kable(q4, digits=4, caption="Q4 Data Output")

```

Table 8: Q4 Data Output

Asset	Start_Date	End_Date	Start_Close	End_Close	Total_Return	Annualized_Return	Mean_Daily_Return	SD_Daily_Return	Observations
BTC	2020-01-02	2025-12-31	6985.47	87508.83	11.5273	0.5244	0.0017	0.0319	2191
AAPL	2020-01-03	2025-12-31	71.63	271.36	2.7884	0.2489	0.0011	0.0200	1507
SPY	2020-01-03	2025-12-31	293.88	678.32	1.3082	0.1498	0.0006	0.0131	1507
BDO	2020-01-03	2025-12-29	128.17	134.60	0.0502	0.0082	0.0003	0.0226	1467

Appendix A11 - SQL Question 5: Best and Worst Trading Days (CRUZ, Ricardo Miguel Iñigo)

Identifies extreme single-day gains and losses for each asset to assess tail risk.

```

# Q5: Best and Worst Trading Days -- CRUZ, Ricardo Miguel Inigo

q5_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q5 <- q5_raw |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  summarise(
    Best_Date = Date[which.max(Daily_Return)],
    Best_Daily_Return = max(Daily_Return, na.rm = TRUE),
    Worst_Date = Date[which.min(Daily_Return)],
    Worst_Daily_Return = min(Daily_Return, na.rm = TRUE),
    Return_Range = Best_Daily_Return - Worst_Daily_Return,
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Worst_Daily_Return)

safe_kable(q5, digits=4, caption="Q5 Data Output")

```

Table 9: Q5 Data Output

Asset	Best_Date	Best_Daily_Return	Worst_Date	Worst_Daily_Return	Return_Range	Observations
BTC	2021-02-08	0.1875	2020-03-12	-0.3717	0.5592	2191
BDO	2020-03-26	0.1570	2020-03-19	-0.2272	0.3842	1467
AAPL	2025-04-09	0.1533	2020-03-16	-0.1286	0.2819	1507
SPY	2025-04-09	0.1050	2020-03-16	-0.1094	0.2144	1507

Appendix A12 - SQL Question 6: Moving Average Crossover Analysis (CRUZ, Ricardo Miguel Iñigo)

Implements a 20-day and 60-day simple moving average crossover strategy to analyze momentum regime shifts.

```
# Q6: Moving Average Crossover Analysis -- CRUZ, Ricardo Miguel Inigo

q6_raw <- fin_data |>
  select(Date, Asset, Close, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q6_ma <- q6_raw |>
  filter(!is.na(Close)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(
    MA_20 = as.numeric(stats::filter(Close, rep(1 / 20, 20), sides = 1)),
    MA_60 = as.numeric(stats::filter(Close, rep(1 / 60, 60), sides = 1)),
    Signal = case_when(
      MA_20 > MA_60 ~ "Bullish",
      MA_20 < MA_60 ~ "Bearish",
      TRUE ~ "Neutral"
    )
  ) |>
  ungroup()

q6 <- q6_ma |>
  filter(!is.na(MA_20), !is.na(MA_60), !is.na(Daily_Return)) |>
  group_by(Asset, Signal) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
```

```

arrange(Asset, desc(Mean_Daily_Return))

q6_latest_signal <- q6_ma |>
  filter(!is.na(MA_20), !is.na(MA_60)) |>
  group_by(Asset) |>
  slice_max(Date, n = 1, with_ties = FALSE) |>
  ungroup() |>
  select(Asset, Date, Close, MA_20, MA_60, Signal) |>
  arrange(Asset)

safe_kable(q6, digits=4, caption="Q6 Data Output")

```

Table 10: Q6 Data Output

Asset	Signal	Mean_Daily_Return	SD_Daily_Return	Observations
AAPL	Bullish	0.0015	0.0168	892
AAPL	Bearish	0.0008	0.0212	557
BDO	Bullish	0.0006	0.0195	667
BDO	Bearish	0.0003	0.0211	742
BTC	Bullish	0.0028	0.0300	1182
BTC	Bearish	0.0002	0.0343	951
SPY	Bearish	0.0013	0.0162	361
SPY	Bullish	0.0006	0.0092	1088

Appendix A13 - SQL Question 7: Volume and Volatility Relationship (CRUZ, Ricardo Miguel Iñigo)

Tests whether elevated trading volume systematically coincides with higher volatility by partitioning days into high-volume and normal-volume regimes.

```

# Q7: Volume and Volatility Relationship -- CRUZ, Ricardo Miguel Inigo

volume_lookup <- assets_all |>
  select(Date, Asset, Volume) |>
  mutate(
    Date = as.Date(Date),
    Volume = as.numeric(Volume)
  )

q7_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(
    Date = as.Date(Date),
    Daily_Return = as.numeric(Daily_Return)
  )

```

```

) |>
left_join(volume_lookup, by = c("Date", "Asset")) |>
filter(
  !is.na(Daily_Return),
  !is.na(Volume),
  Volume > 0
)

q7 <- q7_raw |>
group_by(Asset) |>
mutate(
  Volume_Threshold = quantile(Volume, 0.75, na.rm = TRUE),
  Volume_Regime = if_else(
    Volume >= Volume_Threshold,
    "High Volume Days",
    "Normal Volume Days"
  ),
  Absolute_Return = abs(Daily_Return)
) |>
ungroup() |>
group_by(Asset, Volume_Regime) |>
summarise(
  Observations = n(),
  Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
  Mean_Absolute_Return = mean(Absolute_Return, na.rm = TRUE),
  Volatility = sd(Daily_Return, na.rm = TRUE),
  Average_Volume = mean(Volume, na.rm = TRUE),
  .groups = "drop"
) |>
arrange(Asset, Volume_Regime)

safe_kable(q7, digits=4, caption="Q7 Data Output")

```

Table 11: Q7 Data Output

Asset	Volume_Regime	Observations	Mean_Daily_Return	Mean_Absolute_Return	Volatility	Average_Volume
AAPL	High Volume Days	377	0.0015	0.0240	0.0323	152332404
AAPL	Normal Volume Days	1130	0.0009	0.0105	0.0137	61873902
BDO	High Volume Days	367	-0.0001	0.0230	0.0322	6581172
BDO	Normal Volume Days	1100	0.0004	0.0136	0.0183	2551276
BTC	High Volume Days	548	0.0017	0.0305	0.0450	65153421094
BTC	Normal Volume Days	1643	0.0016	0.0183	0.0261	26930318305
SPY	High Volume Days	377	-0.0029	0.0160	0.0220	129389767
SPY	Normal Volume Days	1130	0.0018	0.0061	0.0078	63711998

Appendix A14 - SQL Question 8: VIX Regime Analysis (SISON, Aaron Joshua Estacio)

Assesses asset performance during periods of high versus low market fear by segregating trading days based on VIX levels.

```
# Q8: VIX Regime Analysis -- SISON, Aaron Joshua Estacio

q8_raw <- fin_data |>
  filter(!is.na(VIX)) |>
  select(Asset, Date, Daily_Return, VIX) |>
  collect() |>
  mutate(Date = as.Date(Date))

q8 <- q8_raw |>
  mutate(
    VIX_Regime = case_when(
      VIX > 30 ~ "High Volatility (VIX > 30)",
      VIX < 20 ~ "Low Volatility (VIX < 20)",
      TRUE ~ "Moderate (20 <= VIX <= 30)"
    )
  ) |>
  group_by(Asset, VIX_Regime) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Asset, VIX_Regime)

q8_plot <- q8 |>
  filter(VIX_Regime %in% c("High Volatility (VIX > 30)", "Low Volatility
    ↵ (VIX < 20)")) |>
  mutate(VIX_Regime = recode(VIX_Regime,
    "High Volatility (VIX > 30)" = "VIX > 30 (High Fear)",
    "Low Volatility (VIX < 20)" = "VIX < 20 (Low Fear)"))

safe_kable(q8, digits=4, caption="Q8 Data Output")
```

Table 12: Q8 Data Output

Asset	VIX_Regime	Mean_Daily_Return	SD_Daily_Return	Observations
AAPL	High Volatility (VIX > 30)	-0.0054	0.0389	148
AAPL	Low Volatility (VIX < 20)	0.0025	0.0134	859
AAPL	Moderate (20 <= VIX <= 30)	0.0007	0.0208	528
BDO	High Volatility (VIX > 30)	-0.0042	0.0411	148
BDO	Low Volatility (VIX < 20)	0.0006	0.0185	859
BDO	Moderate (20 <= VIX <= 30)	0.0011	0.0213	528
BTC	High Volatility (VIX > 30)	0.0016	0.0550	148
BTC	Low Volatility (VIX < 20)	0.0022	0.0288	859
BTC	Moderate (20 <= VIX <= 30)	0.0020	0.0380	528
SPY	High Volatility (VIX > 30)	-0.0044	0.0303	148
SPY	Low Volatility (VIX < 20)	0.0018	0.0069	859
SPY	Moderate (20 <= VIX <= 30)	0.0002	0.0123	528

Appendix A15 - SQL Question 9: Interest Rate Sensitivity (SISON, Aaron Joshua Estacio)

Measures asset sensitivity to changes in the 10-year Treasury yield by comparing mean returns on days when yields rise versus fall.

```
# Q9: Interest Rate Sensitivity -- SISON, Aaron Joshua Estacio
```

```
q9_raw <- fin_data |>
  filter(!is.na(DGS10)) |>
  select(Asset, Date, Daily_Return, DGS10) |>
  collect() |>
  mutate(Date = as.Date(Date))

q9 <- q9_raw |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(DGS10_Change = DGS10 - lag(DGS10)) |>
  ungroup() |>
  filter(!is.na(DGS10_Change)) |>
  mutate(
    Yield_Direction = case_when(
      DGS10_Change > 0 ~ "Yields Rising",
      DGS10_Change < 0 ~ "Yields Falling",
      TRUE ~ "No Change"
    )
  ) |>
  group_by(Asset, Yield_Direction) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
```

```

    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Asset, Yield_Direction)

q9_plot <- q9 |>
  filter(Yield_Direction %in% c("Yields Rising", "Yields Falling"))

safe_kable(q9, digits=4, caption="Q9 Data Output")

```

Table 13: Q9 Data Output

Asset	Yield_Direction	Mean_Daily_Return	SD_Daily_Return	Observations
AAPL	No Change	0.0019	0.0195	103
AAPL	Yields Falling	0.0005	0.0200	684
AAPL	Yields Rising	0.0013	0.0201	712
BDO	No Change	-0.0020	0.0208	103
BDO	Yields Falling	0.0000	0.0253	684
BDO	Yields Rising	0.0007	0.0201	712
BTC	No Change	0.0028	0.0372	103
BTC	Yields Falling	0.0017	0.0343	684
BTC	Yields Rising	0.0020	0.0363	712
SPY	No Change	-0.0001	0.0101	103
SPY	Yields Falling	-0.0001	0.0130	684
SPY	Yields Rising	0.0014	0.0135	712

Appendix A16 - SQL Question 10: Inflation Hedge Performance (SISON, Aaron Joshua Estacio)

Evaluates which assets served as effective inflation hedges during the June 2021 to February 2023 high-inflation period (CPI YoY above 5%).

```
# Q10: Inflation Hedge Performance -- SISON, Aaron Joshua Estacio
```

```

q10_raw <- fin_data |>
  select(Date, Asset, Daily_Return, CPI_YoY) |>
  collect() |>
  mutate(Date = as.Date(Date))

q10 <- q10_raw |>
  filter(!is.na(CPI_YoY) & CPI_YoY > 0.05) |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Cumulative_Return = cumprod(1 + Daily_Return) - 1) |>
  summarise(

```

```

    Start_Date = min(Date),
    End_Date = max(Date),
    Start_CPI_YoY = first(CPI_YoY),
    End_CPI_YoY = last(CPI_YoY),
    Total_Cumulative_Return = last(Cumulative_Return),
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    Observations = n()
  ) |>
  arrange(desc(Total_Cumulative_Return))

q10_plot <- q10 |>
  mutate(Asset = factor(Asset, levels =
    ↪ q10$Asset[order(q10$Total_Cumulative_Return)]))

safe_kable(q10, digits=4, caption="Q10 Data Output")

```

Table 14: Q10 Data Output

Asset	Start_Date	End_Date	Start_CPI_YoY	End_CPI_YoY	Total_Cumulative_Return	Mean_Daily_Return	Observations
BDO	2021-06-01	2023-02-28	0.053	0.0596	0.4391	0.0010	435
AAPL	2021-06-01	2023-02-28	0.053	0.0596	0.1952	0.0006	440
SPY	2021-06-01	2023-02-28	0.053	0.0596	-0.0317	0.0000	440
BTC	2021-06-01	2023-02-28	0.053	0.0596	-0.3800	-0.0002	638

Appendix A17 - Data Visualization Charts (SEBALLOS, Josiah Dweyn Panganiban)

Contains the ggplot code for all 10 figures corresponding to SQL Questions Q1 through Q10. Each figure is saved as a PDF to the reports/figures/ directory for inclusion in the main report's Data Visualization section.

```

# Create figures directory if it does not exist
dir.create("../reports/figures", recursive = TRUE, showWarnings = FALSE)

```

Figure 1: Risk-Adjusted Daily Return by Asset

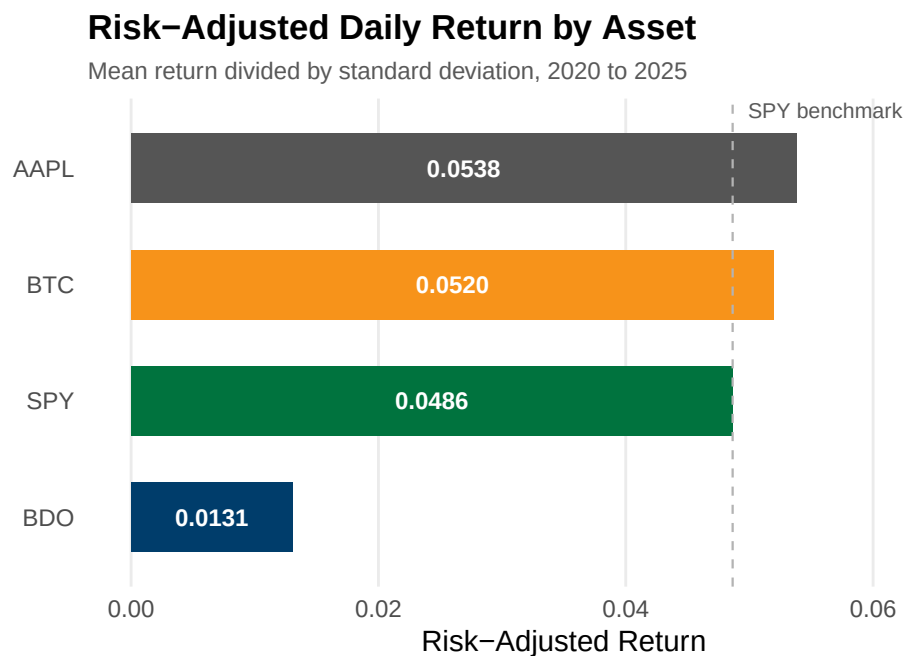
Bar chart showing the ratio of mean daily return to standard deviation for each asset. The dashed reference line marks SPY's ratio.

```

ggplot(q1, aes(x = Risk_Adjusted_Return, y = reorder(Asset,
  ↪ Risk_Adjusted_Return),
               fill = Asset)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_vline(xintercept = spy_ratio, linetype = "dashed",
             color = "#B3B3B3", linewidth = 0.4) +

```

```
geom_text(aes(x = Risk_Adjusted_Return / 2,
              label = sprintf("%.4f", Risk_Adjusted_Return)),
          hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +
annotate("text", x = spy_ratio, y = 4.5, label = "SPY benchmark",
          size = 2.8, color = "#595959", hjust = -0.1) +
scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                              BTC = "#F7931A", SPY = "#00733E")) +
labs(title = "Risk-Adjusted Daily Return by Asset",
     subtitle = "Mean return divided by standard deviation, 2020 to 2025",
     x = "Risk-Adjusted Return", y = NULL) +
xlim(0, 0.07) +
theme_minimal(base_size = 11) +
theme(panel.grid.major.y = element_blank(),
      panel.grid.minor = element_blank(),
      axis.ticks = element_blank(),
      plot.title = element_text(face = "bold", size = 13),
      plot.subtitle = element_text(size = 9, color = "#595959"))
```

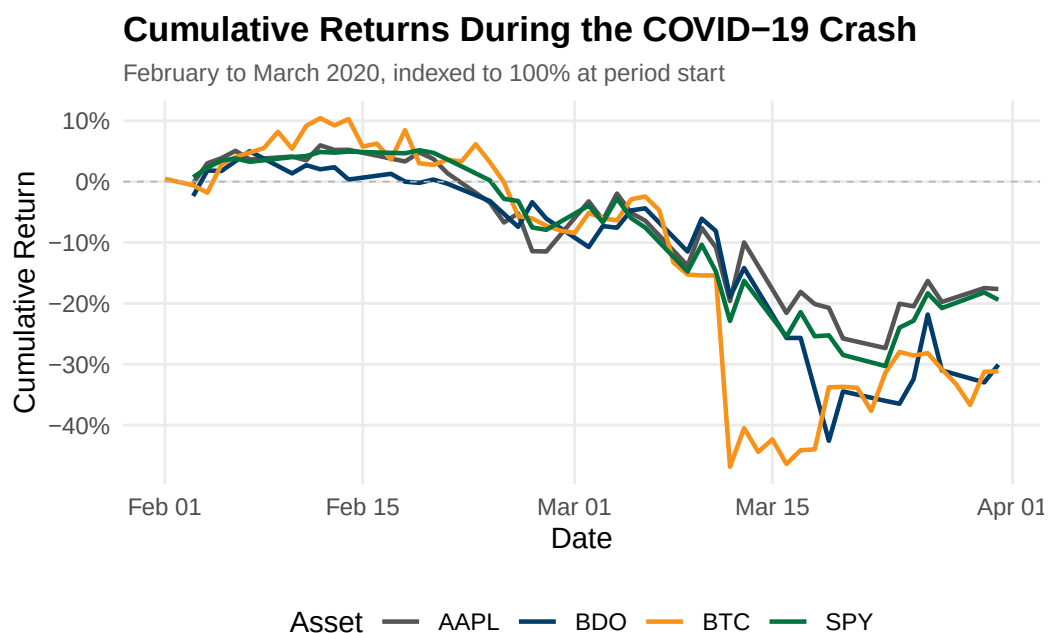


```
ggsave("../reports/figures/fig_q1.pdf", width = 7, height = 3.5, device =
  ↪ "pdf")
```

Figure 2: Cumulative Returns During the COVID-19 Crash

Line chart tracking each asset's cumulative return from February to March 2020, indexed to 100% at period start.

```
ggplot(q2, aes(x = Date, y = Cumulative_Return, color = Asset)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "#B3B3B3",
    ↪ linewidth = 0.3) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
    BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Cumulative Returns During the COVID-19 Crash",
    subtitle = "February to March 2020, indexed to 100% at period start",
    x = "Date", y = "Cumulative Return", color = "Asset") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"),
    legend.position = "bottom")
```



```
ggsave("../reports/figures/fig_q2.pdf", width = 7, height = 4, device =
  ↪ "pdf")
```

Figure 3: Pairwise Correlation Heatmap

Heatmap matrix of Pearson correlation coefficients for daily returns across the four assets.

```
ggplot(cor_long, aes(x = Asset2, y = Asset1, fill = Correlation)) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = sprintf("%.3f", Correlation)), size = 3.5) +
  scale_fill_gradient2(low = "#1DC9A4", mid = "#F2F2F2", high = "#2E45B8",
```

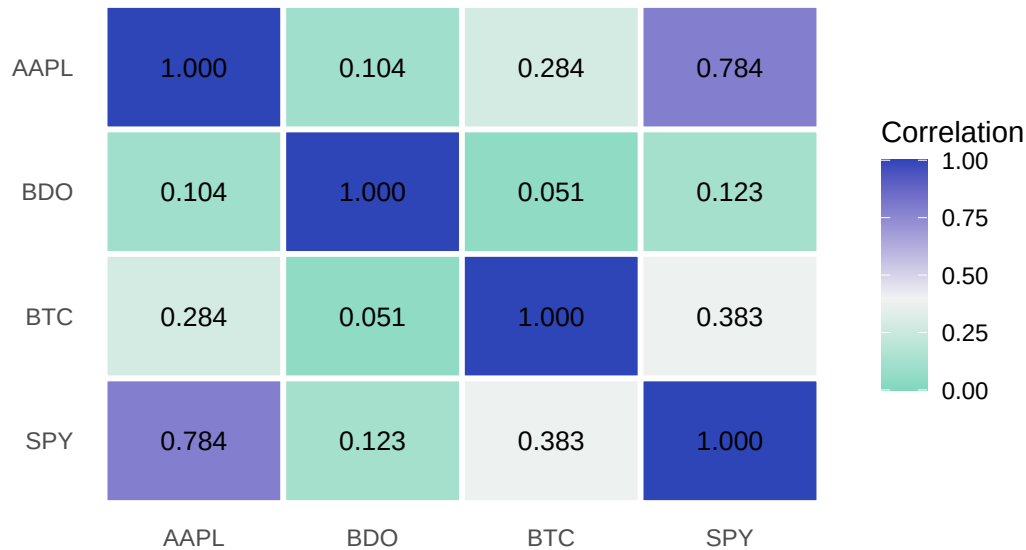
```

midpoint = 0.4, limits = c(0, 1)) +
labs(title = "Pairwise Correlation of Daily Returns",
      subtitle = "Pearson correlation coefficients, 2020 to 2025",
      x = NULL, y = NULL, fill = "Correlation") +
theme_minimal(base_size = 11) +
theme(panel.grid = element_blank(),
      axis.ticks = element_blank(),
      plot.title = element_text(face = "bold", size = 13),
      plot.subtitle = element_text(size = 9, color = "#595959"),
      legend.position = "right")

```

Pairwise Correlation of Daily Returns

Pearson correlation coefficients, 2020 to 2025



```

ggsave("../reports/figures/fig_q3.pdf", width = 6, height = 4.5, device =
  ↪ "pdf")

```

Figure 4: Asset Ranking by Total Return

Horizontal bar chart ranking assets by total percentage price return from 2020 to 2025.

```

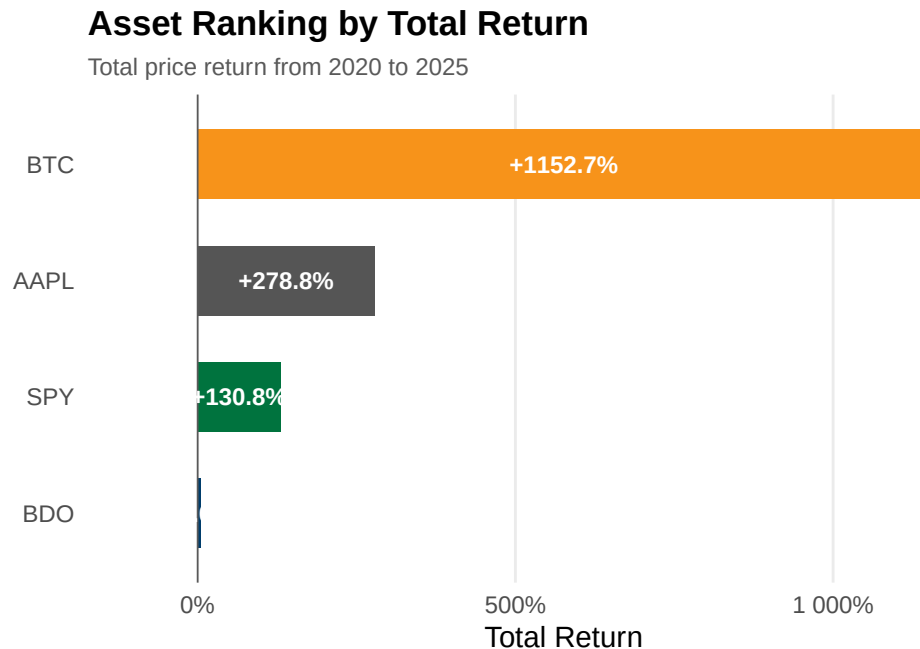
ggplot(q4_plot, aes(x = Total_Return, y = Asset, fill = Asset)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_vline(xintercept = 0, linewidth = 0.3, color = "#595959") +
  geom_text(aes(x = Total_Return / 2,
                label = sprintf("%+.1f%", Total_Return * 100)),
            hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                              BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format(),

```

```

    expand = expansion(mult = c(0.15, 0.15))) +
  labs(title = "Asset Ranking by Total Return",
       subtitle = "Total price return from 2020 to 2025",
       x = "Total Return", y = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.y = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))

```



```

ggsave("../reports/figures/fig_q4.pdf", width = 7, height = 3.5, device =
  ↪ "pdf")

```

Figure 5: Best and Worst Single-Day Returns

Grouped bar chart comparing the largest single-day gain and loss for each asset.

```

q5_plot <- q5 |>
  select(Asset, Best_Daily_Return, Worst_Daily_Return) |>
  pivot_longer(cols = c(Best_Daily_Return, Worst_Daily_Return),
               names_to = "Trading_Day_Type",
               values_to = "Daily_Return") |>
  mutate(
    Trading_Day_Type = recode(Trading_Day_Type,
                             Best_Daily_Return = "Best Trading Day",
                             Worst_Daily_Return = "Worst Trading Day")
  )

```

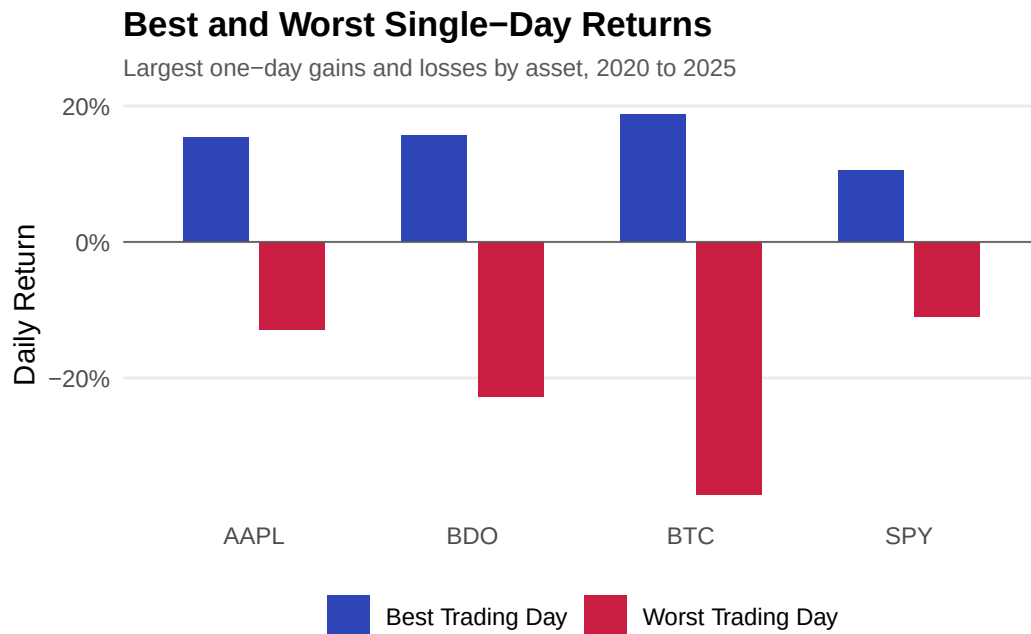


```

)

ggplot(q5_plot, aes(x = Asset, y = Daily_Return, fill = Trading_Day_Type)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +
  scale_fill_manual(values = c("Best Trading Day" = "#2E45B8",
                              "Worst Trading Day" = "#C91D42")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Best and Worst Single-Day Returns",
       subtitle = "Largest one-day gains and losses by asset, 2020 to 2025",
       x = NULL, y = "Daily Return", fill = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")

```



```

ggsave("../reports/figures/fig_q5.pdf", width = 7, height = 4, device =
  ↪ "pdf")

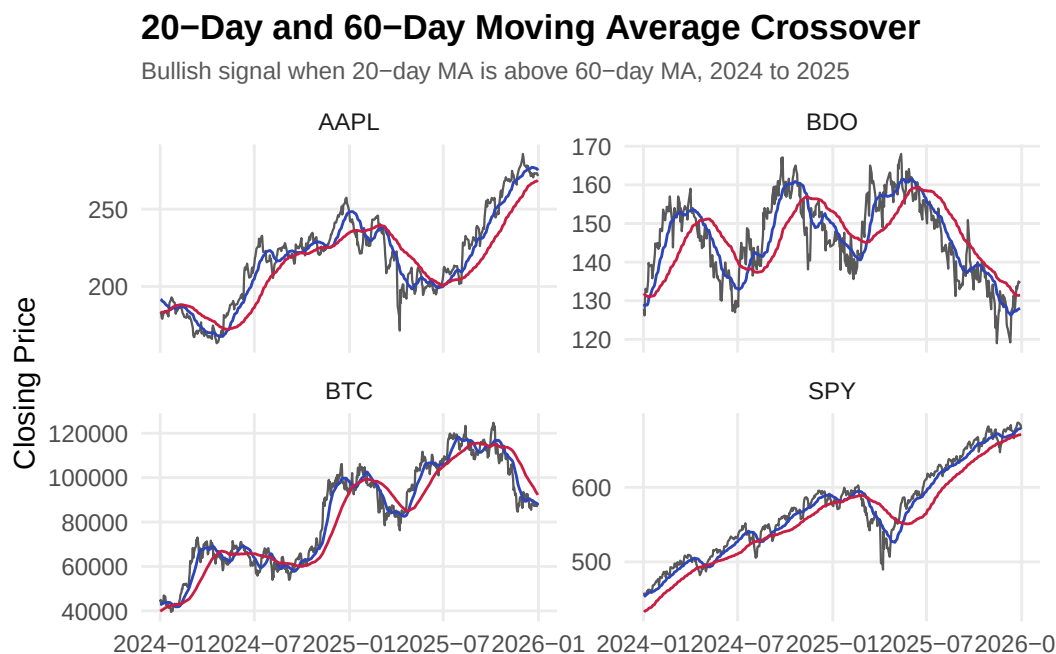
```

Figure 6: 20-Day and 60-Day Moving Average Crossover

Multi-panel faceted line chart showing closing prices with 20-day and 60-day moving averages from 2024 to 2025.

```
q6_plot <- q6_ma |>
  filter(Date >= as.Date("2024-01-01"))

ggplot(q6_plot, aes(x = Date)) +
  geom_line(aes(y = Close), linewidth = 0.4, color = "#595959") +
  geom_line(aes(y = MA_20), linewidth = 0.5, color = "#2E45B8") +
  geom_line(aes(y = MA_60), linewidth = 0.5, color = "#C91D42") +
  facet_wrap(~ Asset, scales = "free_y") +
  labs(title = "20-Day and 60-Day Moving Average Crossover",
       subtitle = "Bullish signal when 20-day MA is above 60-day MA, 2024 to
       ↪ 2025",
       x = NULL, y = "Closing Price") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```



```
ggsave("../reports/figures/fig_q6.pdf", width = 7, height = 4.5, device =
  ↪ "pdf")
```

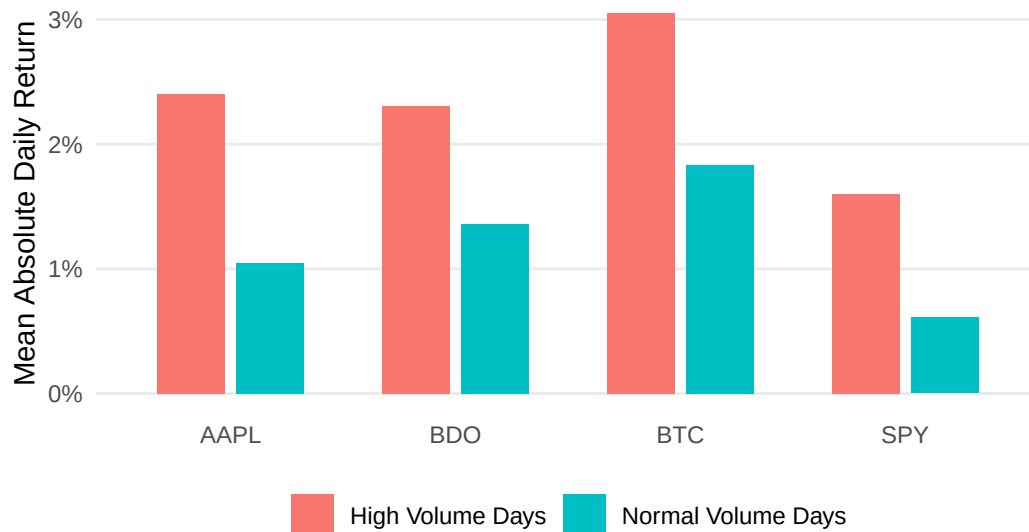
Figure 7: Volume and Volatility Relationship

Grouped bar chart comparing mean absolute daily return during high-volume versus normal-volume days.

```
ggplot(q7, aes(x = Asset, y = Mean_Absolute_Return, fill = Volume_Regime)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "Volume and Volatility Relationship",
    subtitle = "Mean absolute daily return during high-volume versus
      ↪ normal-volume days, 2020 to 2025",
    x = NULL,
    y = "Mean Absolute Daily Return",
    fill = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"),
    legend.position = "bottom"
  )
```

Volume and Volatility Relationship

Mean absolute daily return during high-volume versus normal-volume days, 2020 to 2025

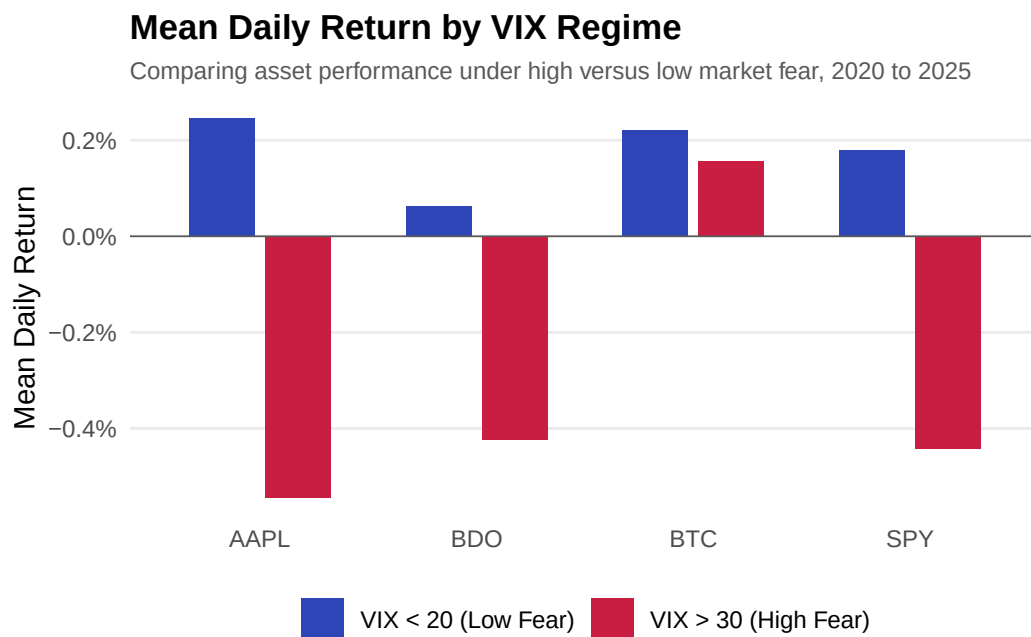


```
ggsave("../reports/figures/fig_q7.pdf", width = 7, height = 4, device =
  ↪ "pdf")
```

Figure 8: Mean Daily Return by VIX Regime

Grouped bar chart comparing asset returns during high-fear ($VIX > 30$) versus low-fear ($VIX < 20$) regimes.

```
ggplot(q8_plot, aes(x = Asset, y = Mean_Daily_Return, fill = VIX_Regime)) +  
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +  
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +  
  scale_fill_manual(values = c("VIX > 30 (High Fear)" = "#C91D42",  
                                "VIX < 20 (Low Fear)" = "#2E45B8")) +  
  scale_y_continuous(labels = scales::percent_format()) +  
  labs(title = "Mean Daily Return by VIX Regime",  
        subtitle = "Comparing asset performance under high versus low market  
        ↪ fear, 2020 to 2025",  
        x = NULL, y = "Mean Daily Return", fill = NULL) +  
  theme_minimal(base_size = 11) +  
  theme(panel.grid.major.x = element_blank(),  
        panel.grid.minor = element_blank(),  
        axis.ticks = element_blank(),  
        plot.title = element_text(face = "bold", size = 13),  
        plot.subtitle = element_text(size = 9, color = "#595959"),  
        legend.position = "bottom")
```

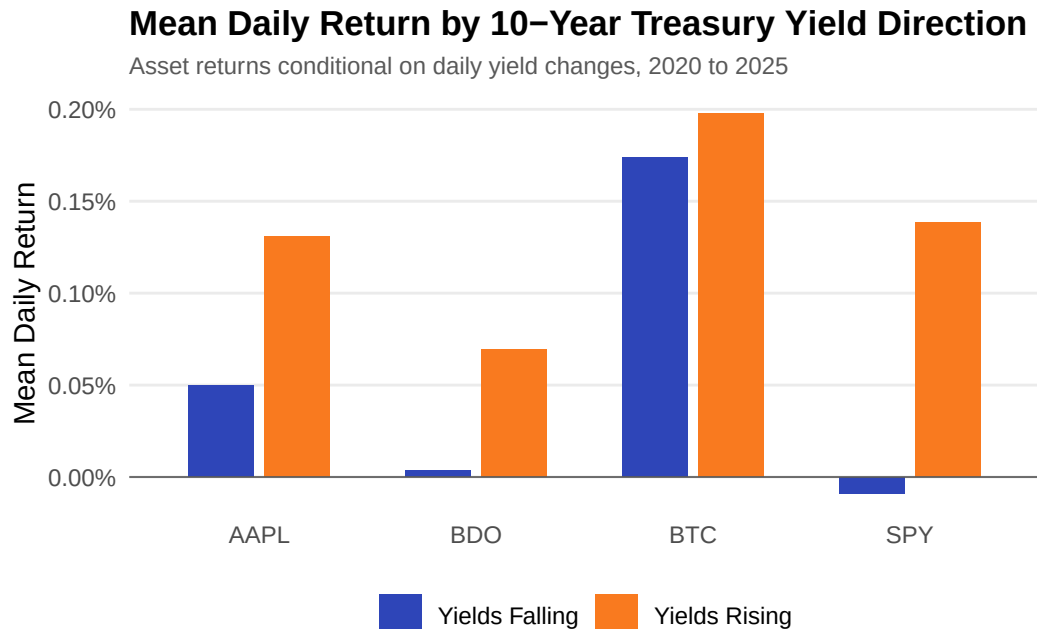


```
ggsave("../reports/figures/fig_q8.pdf", width = 7, height = 4, device =  
  ↪ "pdf")
```

Figure 9: Mean Daily Return by 10-Year Treasury Yield Direction

Grouped bar chart showing asset returns conditional on whether daily yields are rising or falling.

```
ggplot(q9_plot, aes(x = Asset, y = Mean_Daily_Return, fill =  
  ↪ Yield_Direction)) +  
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +  
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +  
  scale_fill_manual(values = c("Yields Rising" = "#F97A1F",  
    "Yields Falling" = "#2E45B8")) +  
  scale_y_continuous(labels = scales::percent_format()) +  
  labs(title = "Mean Daily Return by 10-Year Treasury Yield Direction",  
    subtitle = "Asset returns conditional on daily yield changes, 2020 to  
    ↪ 2025",  
    x = NULL, y = "Mean Daily Return", fill = NULL) +  
  theme_minimal(base_size = 11) +  
  theme(panel.grid.major.x = element_blank(),  
    panel.grid.minor = element_blank(),  
    axis.ticks = element_blank(),  
    plot.title = element_text(face = "bold", size = 13),  
    plot.subtitle = element_text(size = 9, color = "#595959"),  
    legend.position = "bottom")
```

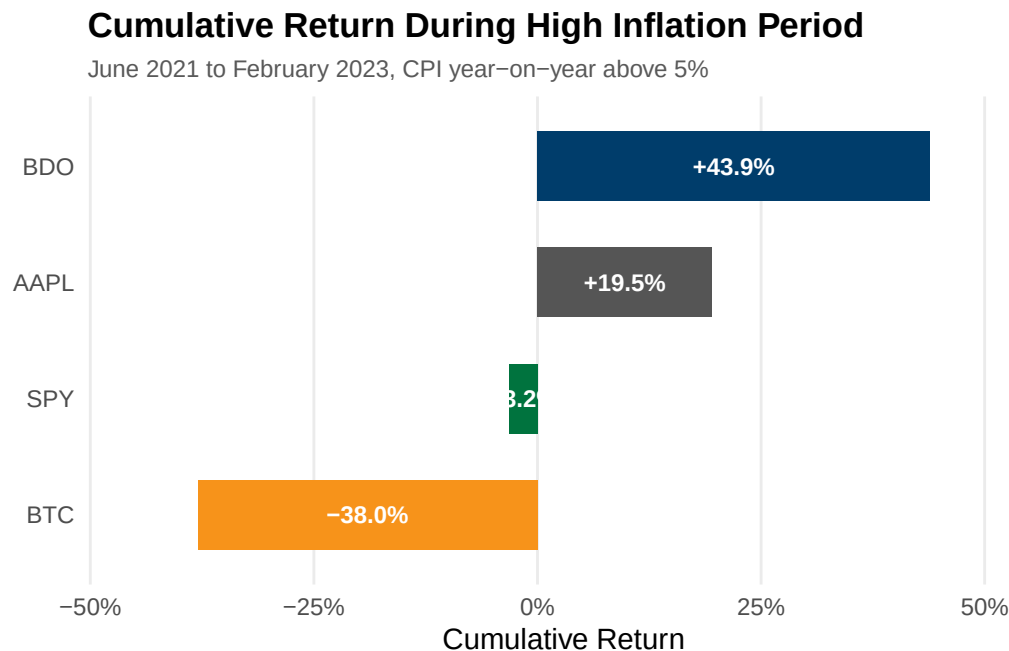


```
ggsave("../reports/figures/fig_q9.pdf", width = 7, height = 4, device =  
  ↪ "pdf")
```

Figure 10: Cumulative Return During High Inflation Period

Horizontal bar chart showing total cumulative returns during the period when CPI year-on-year exceeded 5%.

```
ggplot(q10_plot, aes(x = Total_Cumulative_Return, y = Asset, fill = Asset))
  +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_text(aes(x = Total_Cumulative_Return / 2,
                label = sprintf("%+.1f%", Total_Cumulative_Return * 100)),
            hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                                BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format(),
                    expand = expansion(mult = c(0.15, 0.15))) +
  labs(title = "Cumulative Return During High Inflation Period",
       subtitle = "June 2021 to February 2023, CPI year-on-year above 5%",
       x = "Cumulative Return", y = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.y = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```



```
ggsave("../reports/figures/fig_q10.pdf", width = 7, height = 3.5, device =
  "pdf")
```

Appendix A18 - Exploratory Data Visualization (SE-BALLOS, Josiah Dweyn Panganiban)

Contains 8 exploratory figures covering cumulative returns, price trends, return distributions, rolling volatility, drawdown analysis, risk-return profiles, and portfolio risk contribution. These complement the SQL-driven charts in A17.

Viz Figure 1: Cumulative Returns by Asset

Tracks the growth of USD 1 invested in each asset from 2020 to 2025, indexed to 100% at the start.

```
fig1_data <- sql_long |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Cumulative_Return = cumprod(1 + Daily_Return) - 1) |>
  ungroup()

ggplot(fig1_data, aes(x = Date, y = Cumulative_Return, color = Asset)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "#B3B3B3",
    ↪ linewidth = 0.3) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
    BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Viz 1: Cumulative Returns by Asset",
    subtitle = "Growth of $1 invested, 2020 to 2025",
    x = "Date", y = "Cumulative Return", color = "Asset") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"),
    legend.position = "bottom")
```

Viz 1: Cumulative Returns by Asset

Growth of \$1 invested, 2020 to 2025



```
ggsave("../reports/fig_viz1.pdf", width = 7, height = 4, device = "pdf")
```

Viz 1: Cumulative Returns by Asset tracks the growth of USD 1 invested in each asset from 2020 to 2025. BTC shows the steepest and most volatile climb, reaching cumulative returns above 1,000% with sharp cyclical peaks and drawdowns. AAPL and SPY both trend steadily upward, with AAPL consistently outperforming the SPY benchmark. BDO remains close to the 0% line for most of the period, indicating limited net nominal gain relative to the other three assets.

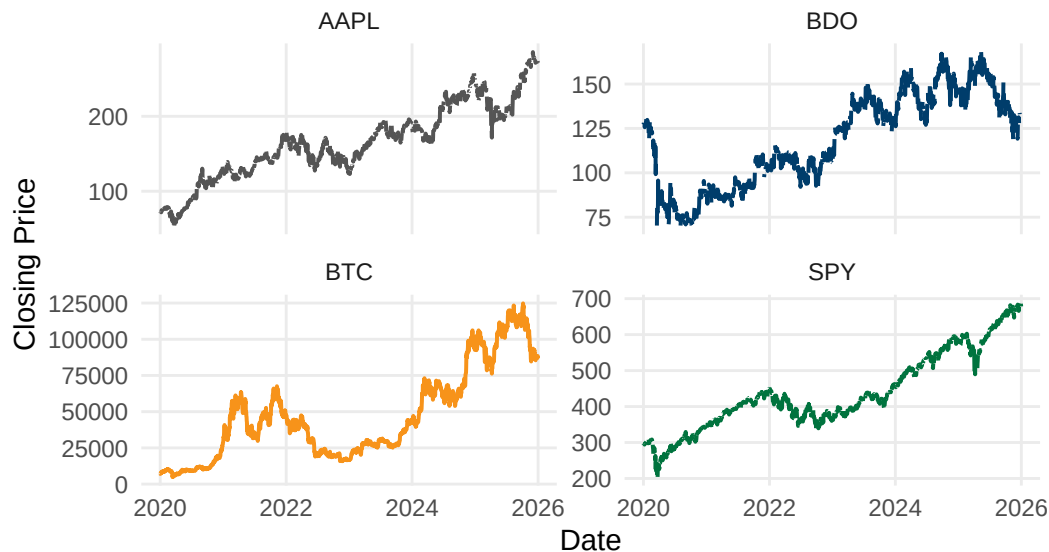
Viz Figure 2: Closing Price Trends by Asset

Plots nominal closing prices on individual scales to compare price-level dynamics across assets of different magnitudes.

```
ggplot(sql_long, aes(x = Date, y = Close, color = Asset)) +  
  geom_line(linewidth = 0.7, show.legend = FALSE, na.rm = TRUE) +  
  facet_wrap(~ Asset, scales = "free_y", ncol = 2) +  
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",  
                                BTC = "#F7931A", SPY = "#00733E")) +  
  labs(title = "Viz 2: Closing Price Trends by Asset",  
       subtitle = "2020 to 2025",  
       x = "Date", y = "Closing Price") +  
  theme_minimal(base_size = 11) +  
  theme(panel.grid.minor = element_blank(),  
        axis.ticks = element_blank(),  
        plot.title = element_text(face = "bold", size = 13),  
        plot.subtitle = element_text(size = 9, color = "#595959"))
```


Viz 2: Closing Price Trends by Asset

2020 to 2025



```
ggsave("../reports/fig_viz2.pdf", width = 7, height = 5, device = "pdf")
```

Viz 2: Closing Price Trends by Asset plots nominal closing prices rather than normalized returns, revealing that BDO experienced meaningful price swings on its own scale despite appearing flat in Viz 1. Each asset uses its own y-axis since raw prices are not directly comparable across magnitudes.

Viz Figure 3: Return Distribution with Skewness and Kurtosis

Overlays each asset's daily return histogram against a fitted normal curve to assess distribution shape and tail risk.

```
fig3_data <- sql_long |> filter(!is.na(Daily_Return))

calc_skew <- function(x) {
  n <- length(x); m <- mean(x)
  (sum((x - m)^3) / n) / (sd(x)^3)
}

calc_kurt <- function(x) {
  n <- length(x); m <- mean(x)
  (sum((x - m)^4) / n) / (sd(x)^2)^2 - 3
}

fig3_stats <- fig3_data |>
  group_by(Asset) |>
  summarise(
    Mean = mean(Daily_Return), SD = sd(Daily_Return),
    Skewness = calc_skew(Daily_Return), Kurtosis = calc_kurt(Daily_Return),
```

```

    .groups = "drop"
  ) |>
  mutate(label = sprintf("Skew: %.2f | Kurt: %.2f", Skewness, Kurtosis))

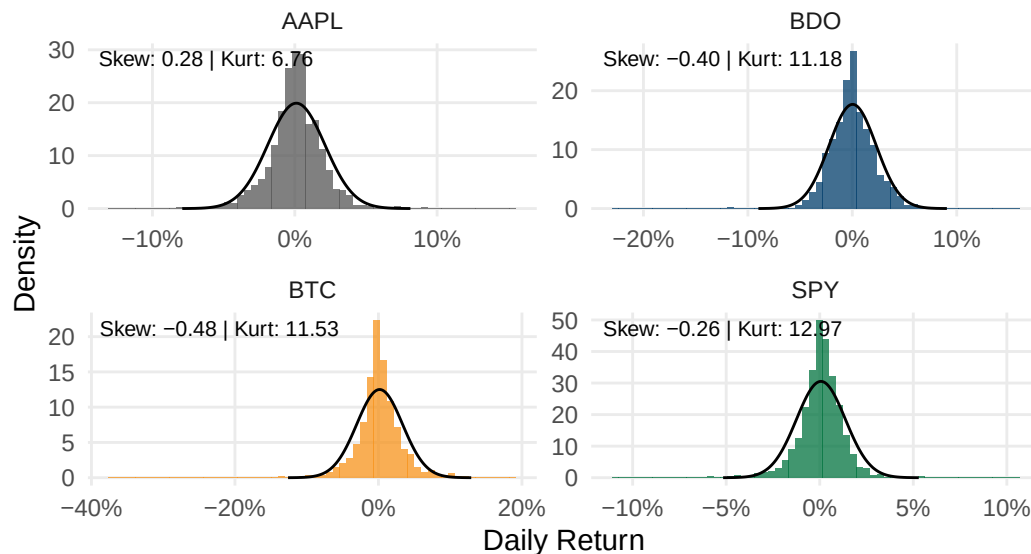
normal_curves <- fig3_stats |>
  rowwise() |>
  reframe(
    Asset = Asset,
    x = seq(Mean - 4 * SD, Mean + 4 * SD, length.out = 200),
    y = dnorm(x, mean = Mean, sd = SD)
  )

ggplot(fig3_data, aes(x = Daily_Return)) +
  geom_histogram(aes(y = after_stat(density), fill = Asset), bins = 60,
    alpha = 0.75, show.legend = FALSE) +
  geom_line(data = normal_curves, aes(x = x, y = y), color = "black",
    ↪ linewidth = 0.5) +
  geom_text(data = fig3_stats, aes(x = -Inf, y = Inf, label = label),
    hjust = -0.05, vjust = 1.5, size = 2.8) +
  facet_wrap(~ Asset, scales = "free", ncol = 2) +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
    BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format()) +
  labs(title = "Viz 3: Return Distribution with Skewness & Kurtosis",
    subtitle = "Daily returns vs. fitted normal distribution (black
    ↪ line), 2020 to 2025",
    x = "Daily Return", y = "Density") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"))

```

Viz 3: Return Distribution with Skewness & Kurtosis

Daily returns vs. fitted normal distribution (black line), 2020 to 2025



```
ggsave("../reports/fig_viz3.pdf", width = 7, height = 5, device = "pdf")
```

Viz 3: Return Distribution with Skewness and Kurtosis overlays each asset's daily return distribution against a fitted normal curve. AAPL is the only asset with positive skewness (0.28), meaning its rare extreme days lean toward the upside. BTC and BDO both show negative skewness (-0.48 and -0.40 respectively), indicating tail risk weighted toward sudden losses. SPY records the highest kurtosis of all four assets (12.97) despite a comparatively mild skew (-0.26), meaning its returns are usually tightly clustered near zero but still capable of extreme shock days. All four kurtosis values exceed 3, the benchmark for a normal distribution, confirming none of these assets follow a standard bell curve.

Viz Figure 4: Return Range and Spread by Asset

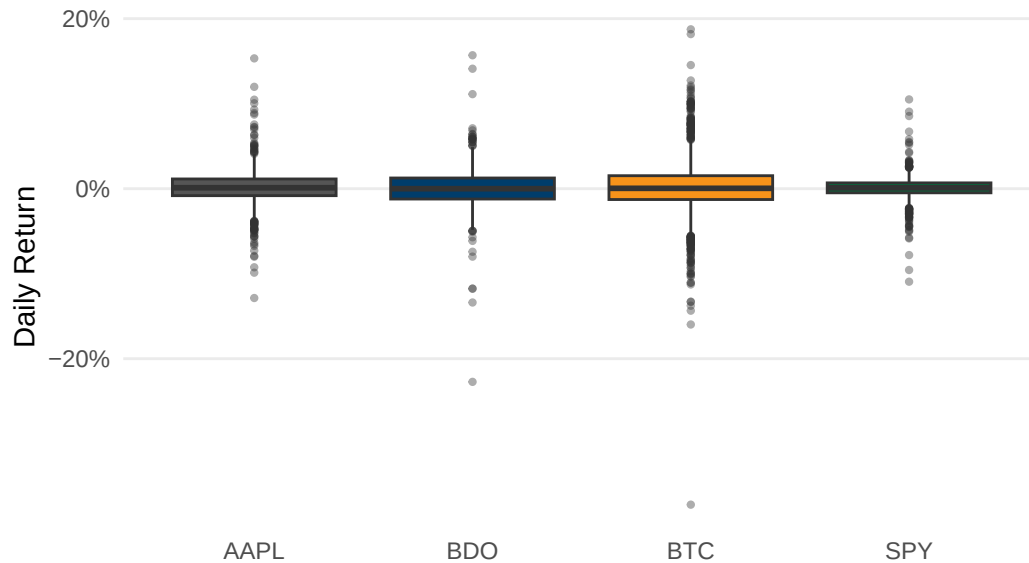
Boxplot showing median, interquartile range, and outlier days for each asset.

```
ggplot(fig3_data, aes(x = Asset, y = Daily_Return, fill = Asset)) +  
  geom_boxplot(show.legend = FALSE, outlier.size = 0.8, outlier.alpha = 0.4)  
  +  
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",  
                                BTC = "#F7931A", SPY = "#00733E")) +  
  scale_y_continuous(labels = scales::percent_format()) +  
  labs(title = "Viz 4: Return Range & Spread by Asset",  
        subtitle = "Median, interquartile range, and outlier days, 2020 to  
        2025",  
        x = NULL, y = "Daily Return") +  
  theme_minimal(base_size = 11) +  
  theme(panel.grid.major.x = element_blank(),  
        panel.grid.minor = element_blank(),
```

```
axis.ticks = element_blank(),
plot.title = element_text(face = "bold", size = 13),
plot.subtitle = element_text(size = 9, color = "#595959"))
```

Viz 4: Return Range & Spread by Asset

Median, interquartile range, and outlier days, 2020 to 2025



```
ggsave("../reports/fig_viz4.pdf", width = 7, height = 4, device = "pdf")
```

Viz 4: Return Range and Spread by Asset shows each asset's typical daily return range and outlier days. BTC has the widest box and most extreme outliers in both directions, consistent with its high kurtosis in Viz 3. SPY has the narrowest, most compressed box, reflecting diversification across 500 companies. AAPL and BDO fall in between, with BDO showing a visible skew toward downside outlier days.

Viz Figure 5: 30-Day Rolling Volatility by Asset

Tracks rolling 30-day standard deviation of daily returns to visualize volatility evolution over time.

```
rolling_sd <- function(x, n = 30) {
  sapply(seq_along(x), function(i) {
    if (i < n) return(NA)
    sd(x[(i - n + 1):i], na.rm = TRUE)
  })
}

fig5_data <- sql_long |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
```

```

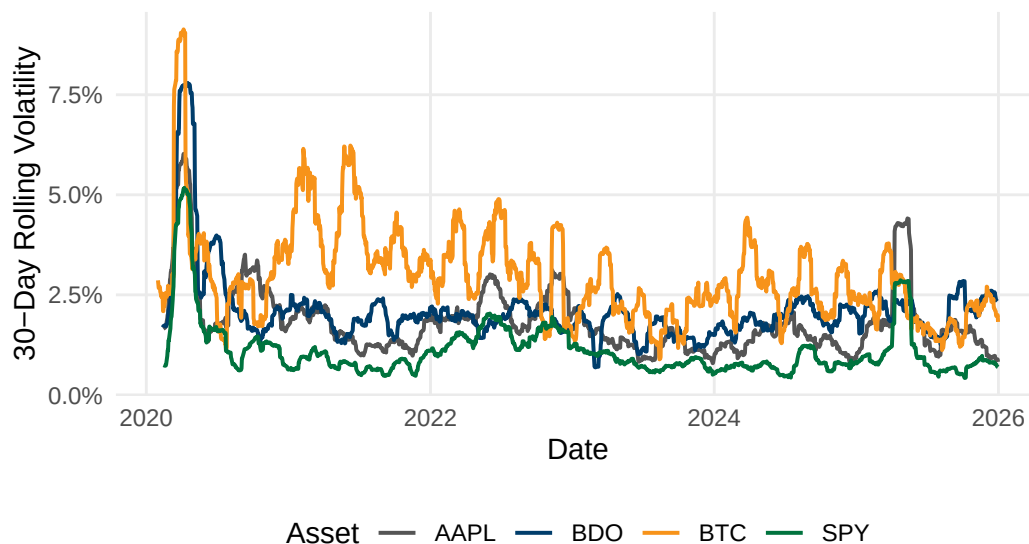
mutate(Rolling_Vol = rolling_sd(Daily_Return, 30)) |>
ungroup() |>
filter(!is.na(Rolling_Vol))

ggplot(fig5_data, aes(x = Date, y = Rolling_Vol, color = Asset)) +
  geom_line(linewidth = 0.7) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                                BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Viz 5: 30-Day Rolling Volatility by Asset",
       subtitle = "Rolling standard deviation of daily returns, 2020 to
       ↵ 2025",
       x = "Date", y = "30-Day Rolling Volatility", color = "Asset") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")

```

Viz 5: 30-Day Rolling Volatility by Asset

Rolling standard deviation of daily returns, 2020 to 2025



```

ggsave("../reports/fig_viz5.pdf", width = 7, height = 4, device = "pdf")

```

Viz 5: 30-Day Rolling Volatility by Asset shows a sharp synchronized volatility spike across all assets in early 2020 from the COVID-19 market shock. BTC consistently maintains the highest rolling volatility baseline with visible cyclical waves from its bull and bear cycles. SPY maintains the lowest and most stable volatility floor. AAPL and BDO show occasional idiosyncratic spikes tied to specific events rather than broad macro shocks.

Viz Figure 6: Drawdown from Peak by Asset

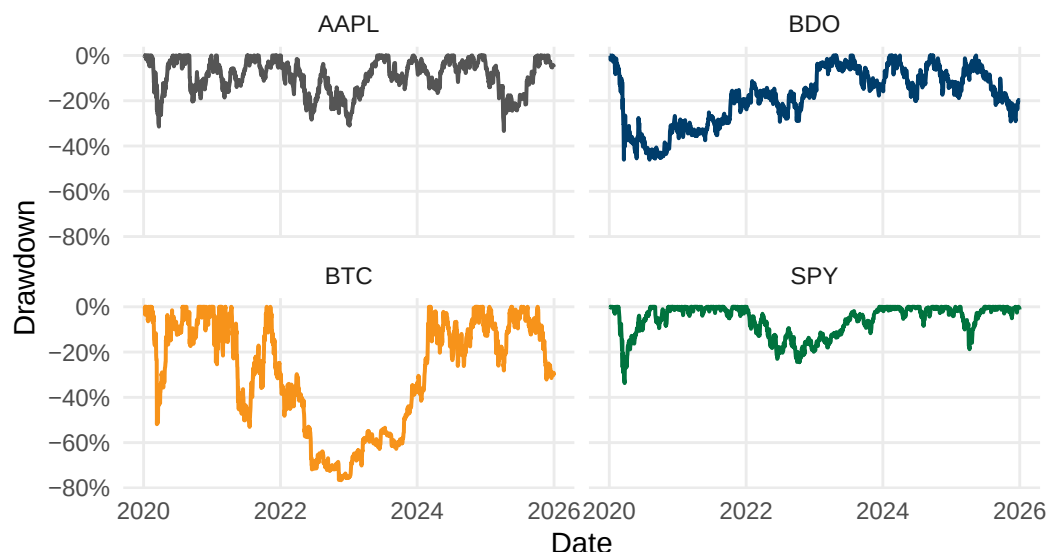
Computes percentage decline from each asset's running historical high to visualize recovery patterns.

```
fig6_data <- fig1_data |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Running_Max = cummax(1 + Cumulative_Return),
         Drawdown = (1 + Cumulative_Return) / Running_Max - 1) |>
  ungroup()

ggplot(fig6_data, aes(x = Date, y = Drawdown, color = Asset)) +
  geom_line(linewidth = 0.7) +
  facet_wrap(~ Asset, ncol = 2) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                                BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Viz 6: Drawdown from Peak by Asset",
       subtitle = "Percentage decline from running maximum value, 2020 to 2025",
       x = "Date", y = "Drawdown") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        legend.position = "none",
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```

Viz 6: Drawdown from Peak by Asset

Percentage decline from running maximum value, 2020 to 2025



```
ggsave("../reports/fig_viz6.pdf", width = 7, height = 5, device = "pdf")
```

Viz 6: Drawdown from Peak by Asset shows each asset's percentage decline from its running historical high. BTC shows the deepest drawdowns falling well below -60% and takes longest to recover to a prior peak. SPY shows the shallowest fastest-recovering drawdowns consistent with its diversified benchmark role. BDO shows a prolonged period in negative territory indicating slower capital recovery through a downturn.

Viz Figure 7: Risk vs. Return by Asset

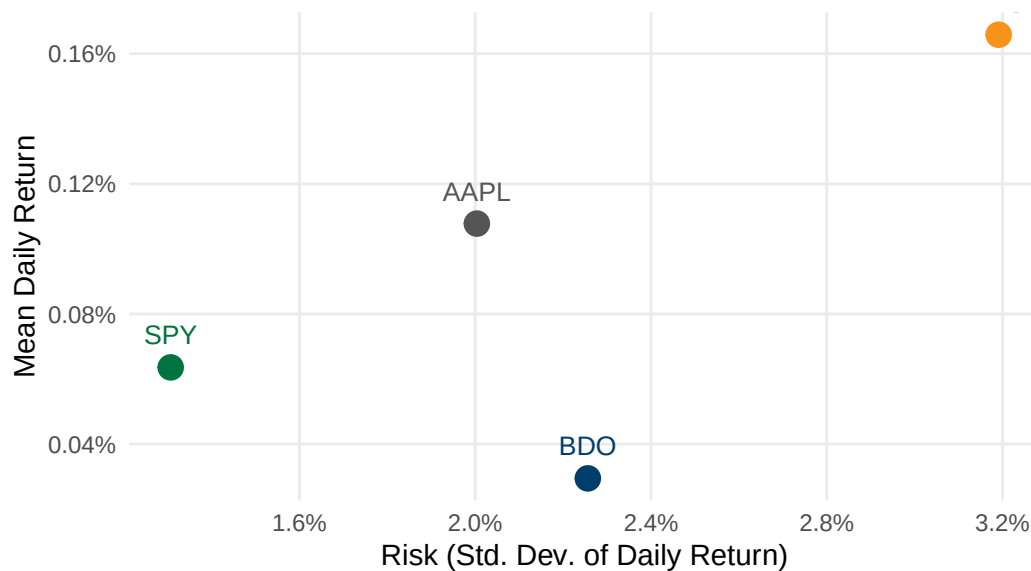
Scatter plot comparing mean daily return against standard deviation across all four assets.

```
fig7_data <- sql_long |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  summarise(Mean_Return = mean(Daily_Return), SD_Return = sd(Daily_Return),
    ↪ .groups = "drop")

ggplot(fig7_data, aes(x = SD_Return, y = Mean_Return, color = Asset)) +
  geom_point(size = 4) +
  geom_text(aes(label = Asset), vjust = -1.2, size = 3.5, show.legend =
    ↪ FALSE) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
    ↪ BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format()) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Viz 7: Risk vs. Return by Asset",
    subtitle = "Mean daily return vs. standard deviation, 2020 to 2025",
    x = "Risk (Std. Dev. of Daily Return)", y = "Mean Daily Return") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    legend.position = "none",
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"))
```

Viz 7: Risk vs. Return by Asset

Mean daily return vs. standard deviation, 2020 to 2025



```
ggsave("../reports/fig_viz7.pdf", width = 7, height = 5, device = "pdf")
```

Viz 7: Risk vs. Return by Asset plots each asset's mean daily return against standard deviation. BTC delivers the highest mean return (0.166%) but also highest risk (3.19% daily SD). AAPL offers a more efficient middle ground with 0.108% mean return at only 2.00% volatility. SPY sits at the lowest-risk end (1.31% SD) with a modest 0.0636% mean return. BDO is the weakest performer on a risk-adjusted basis, carrying similar risk to AAPL (2.26% SD) while delivering the lowest mean return (0.0295%).

Viz Figure 8: Equal-Weighted Portfolio Risk Contribution

Decomposes total portfolio variance into each asset's contribution to show that equal dollar weights do not produce equal risk.

```
returns_matrix <- sql_long |>
  filter(Asset %in% c("AAPL", "BDO", "BTC"), !is.na(Daily_Return)) |>
  select(Date, Asset, Daily_Return) |>
  pivot_wider(names_from = Asset, values_from = Daily_Return) |>
  select(AAPL, BDO, BTC) |>
  na.omit()

cov_matrix <- cov(returns_matrix)
weights <- c(AAPL = 1/3, BDO = 1/3, BTC = 1/3)
port_var <- as.numeric(t(weights) %*% cov_matrix %*% weights)
marginal_contrib <- cov_matrix %*% weights
risk_contrib <- as.numeric(weights * marginal_contrib / port_var)

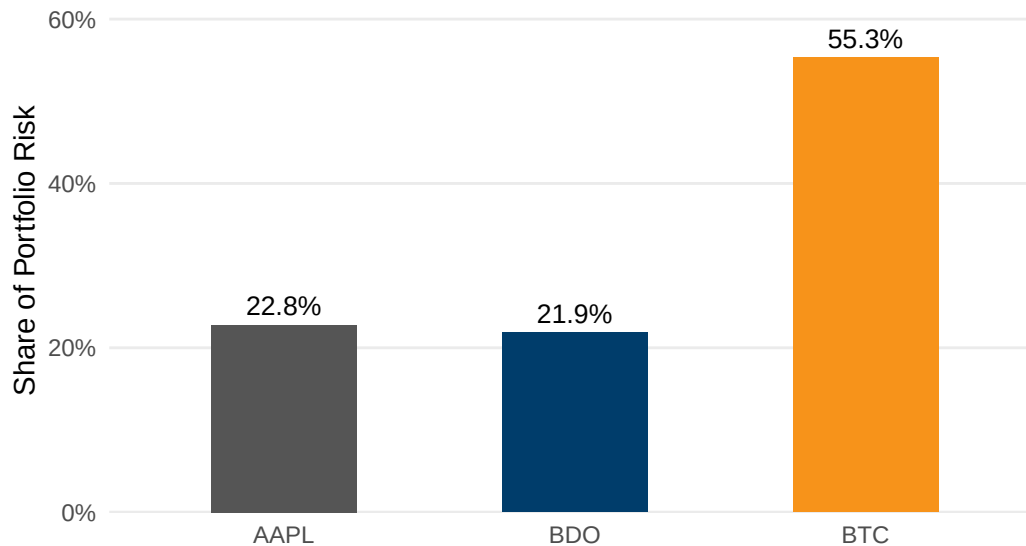
fig8_data <- data.frame(Asset = names(weights), Risk_Contribution =
  ↪ risk_contrib)
```



```
ggplot(fig8_data, aes(x = Asset, y = Risk_Contribution, fill = Asset)) +
  geom_col(width = 0.5, show.legend = FALSE) +
  geom_text(aes(label = scales::percent(Risk_Contribution, accuracy = 0.1)),
    vjust = -0.5, size = 3.5) +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B", BTC =
    ↪ "#F7931A")) +
  scale_y_continuous(labels = scales::percent_format(), expand =
    ↪ expansion(mult = c(0, 0.15))) +
  labs(title = "Viz 8: Equal-Weighted Portfolio Risk Contribution",
    subtitle = "Share of total portfolio variance contributed by each
    ↪ asset",
    x = NULL, y = "Share of Portfolio Risk") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"))
```

Viz 8: Equal-Weighted Portfolio Risk Contribution

Share of total portfolio variance contributed by each asset



```
ggsave("../reports/fig_viz8.pdf", width = 7, height = 4, device = "pdf")
```

Viz 8: Equal-Weighted Portfolio Risk Contribution decomposes total portfolio variance under an equal one-third allocation. BTC alone contributes 55.3% of total portfolio variance despite representing only one-third of invested capital. AAPL and BDO contribute 22.8% and 21.9% respectively, roughly proportional to their capital weights. Equal-dollar allocation does not produce equal-risk allocation, supporting a reduced BTC weight to lower portfolio risk

without proportional loss in expected return.

Sec 7: Descriptive Statistics – GALEDO, Enrique Lorenzo Hermoso

Mean, median, variance, std dev, min, max, range, skewness, kurtosis.

```
# Sec 7: Descriptive Statistics -- compute mean, median, variance, skewness,  
↪ kurtosis -- GALEDO, Enrique Lorenzo Hermoso
```

Sec 8: Data Visualization – SEBALLOS, Josiah Dweyn Panganiban

At least 8 professional-quality figures.

```
# Sec 8: Data Visualization -- create 8 professional figures with ggplot2 --  
↪ SEBALLOS, Josiah Dweyn Panganiban
```

Sec 9: Exploratory Analysis – SEECHUNG, Camille Castro

Compare returns, volatility, inflationary periods, high-VIX periods, interest-rate environments.

```
# Sec 9: Exploratory Financial Analysis -- compare returns, volatility,  
↪ inflation, VIX, interest rates -- SEECHUNG, Camille Castro
```

Sec 10: Investment Recommendation – SEECHUNG, Camille Castro

Allocate USD 1,000,000.

```
# Sec 10: Investment Recommendation -- USD 1,000,000 allocation among AAPL,  
↪ BDO, BTC, cash -- SEECHUNG, Camille Castro
```

Sec 11: Executive Dashboard – CRUZ, Ricardo Miguel Iñigo

```
# Sec 11: Executive Dashboard -- key stats, charts, SQL findings,  
↪ recommendation summary -- CRUZ, Ricardo Miguel Iñigo
```